# CS3213: Foundations of Software Engineering

Property-based Testing

© Copyright National University of Singapore. All Rights Reserved.

# Differential Testing

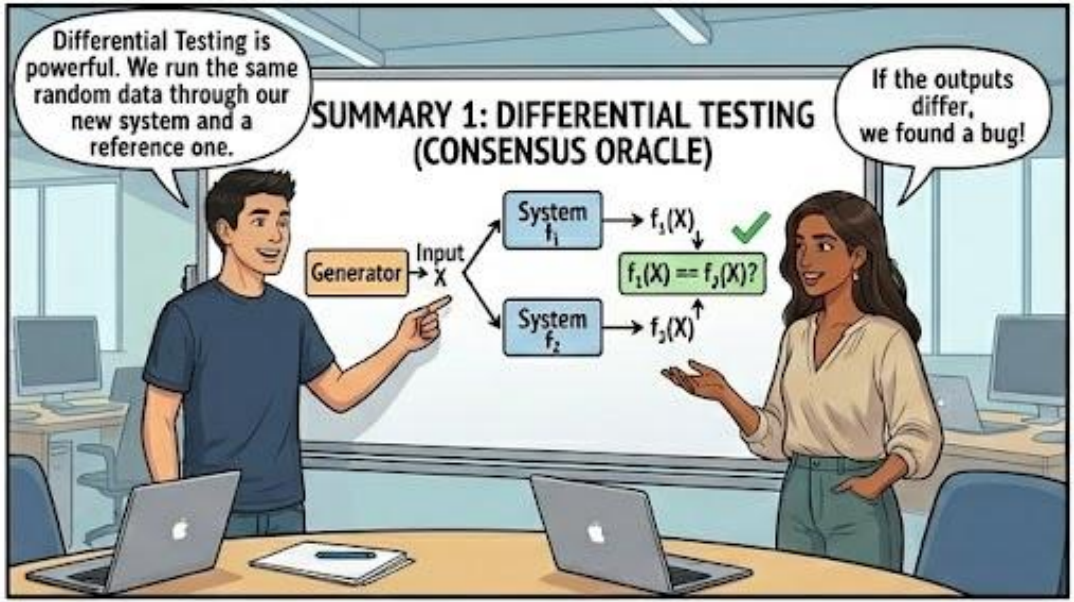



Image by Gemini

# Metamorphic Testing

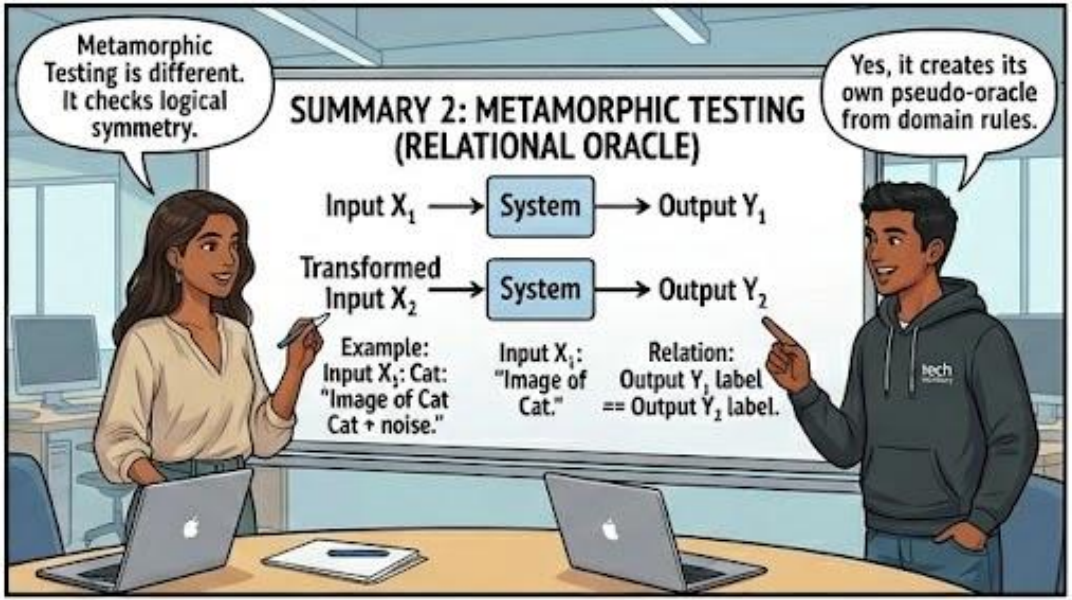



Image by Gemini

# Generalizing Them?

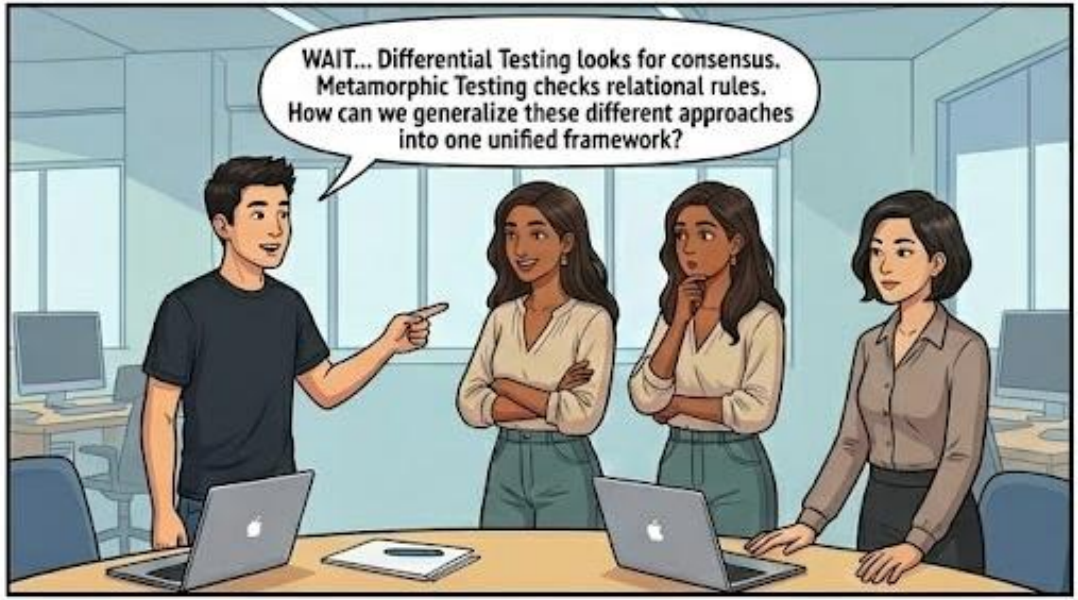



Image by Gemini

# Property-based Testing!

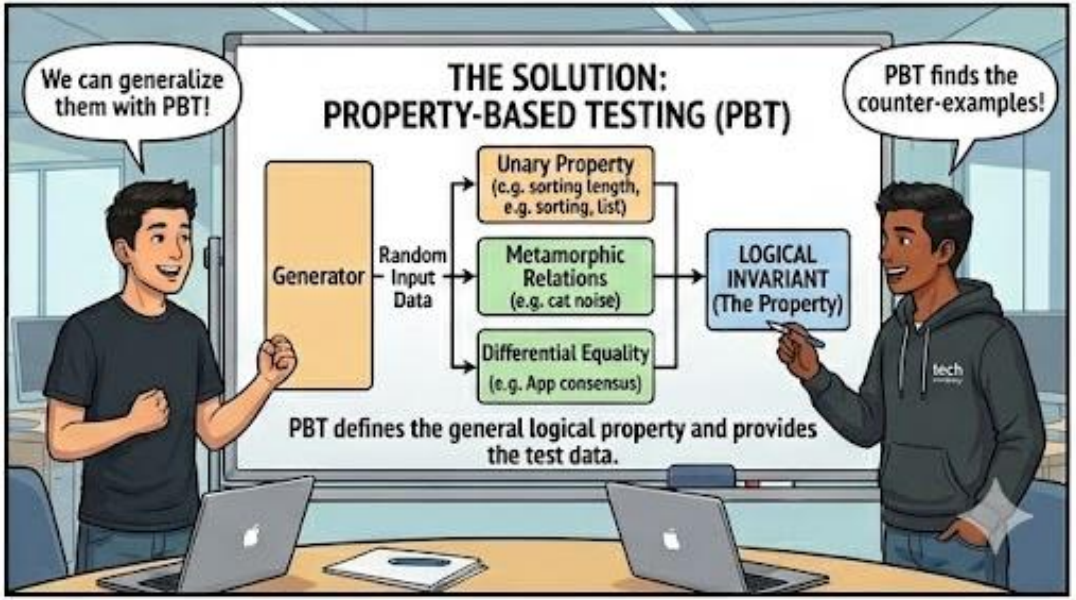



Image by Gemini

# Property-based Testing!

```
assertThat(list.size()).isGreaterThanOrEqualTo(0);
```

# Property-based Testing (PBT)

- **Property-based testing**: specify properties for the component we can test and let the test framework try to find a counterexample that causes the property to break (i.e., falsify it)

# Test Oracle + Test Input

Test oracle is the *property* specified by us

Test Oracle + Test Input

# Test Oracle + Test Input

Test oracle is the *property* specified by us

The test input is generated by a framework (e.g., based on the input data types or by a *generator* specified by us)

Test Oracle + Test Input

# Property-based Testing Illustration

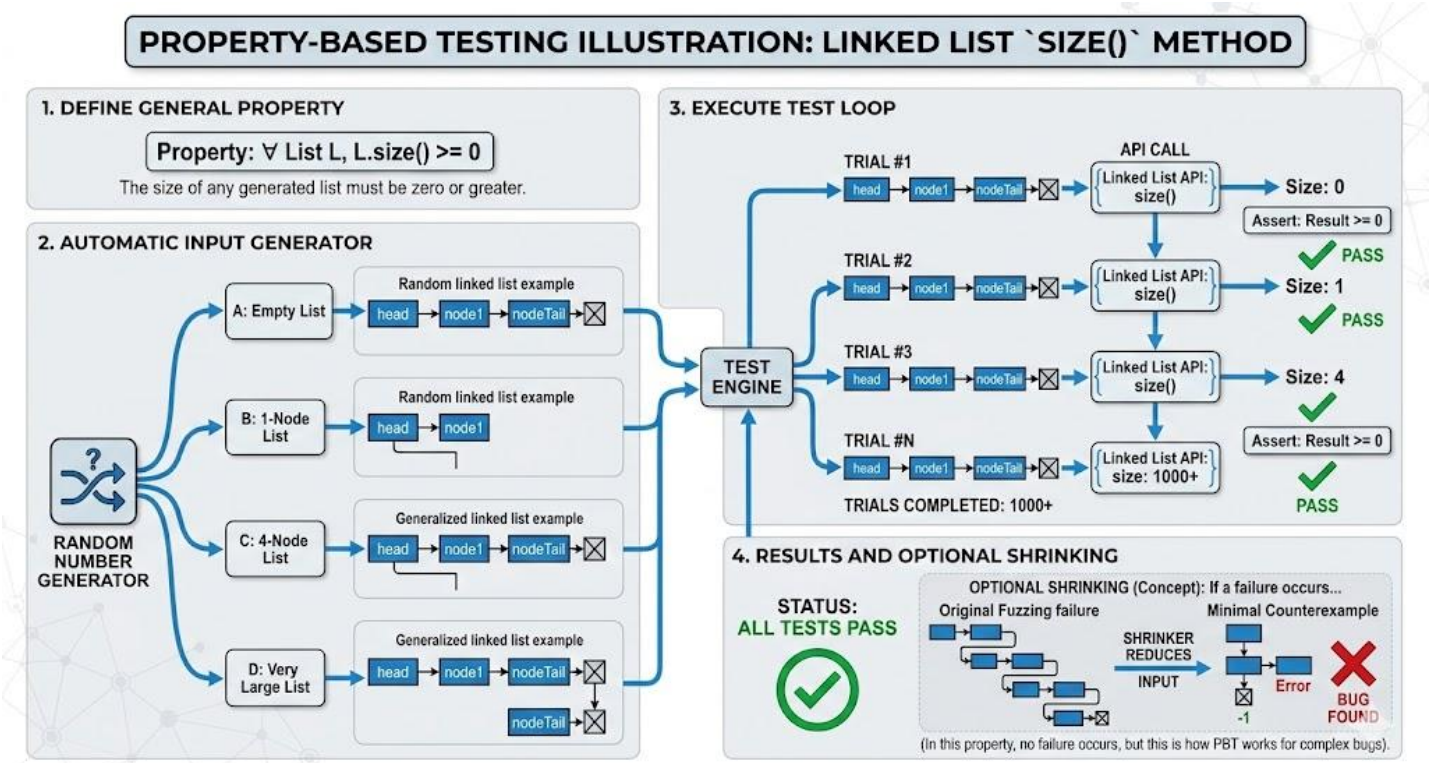
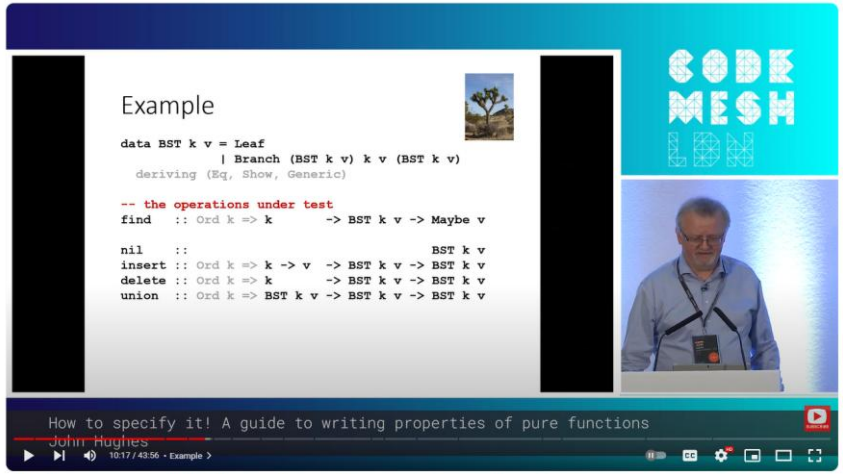



Image by Gemini

# Property-based Testing

- Pioneered in QuickCheck for Haskell
  - Implemented in ~300 lines of code




John Hughes - How to specify it! A guide to writing properties of pure functions | Code Mesh LDN 19

https://www.youtube.com/watch?v=zvRAyq5wj38

## QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs


Koen Claessen
Chalmers University of Technology
koen@cs.chalmers.se

John Hughes
Chalmers University of Technology
rjmh@cs.chalmers.se



### ABSTRACT

QuickCheck is a tool which aids the Haskell programmer in formulating and testing properties of programs. Properties are described as Haskell functions, and can be automatically tested on random input, but it is also possible to define custom test data generators. We present a number of case studies, in whic hthe tool w assuccessfully used, and also point out some pitfalls to avoid. Random testing is especially suitable for functional programs because properties can be stated at a fine grain. When a function is built from separately tested components, then random testing suffices to obtain good coverage of the definition under test.


### 1. INTRODUCTION

Testing is by far the most commonly used approach to ensuring softw are qualit y. It is also very labour intensive, accoun tingfor up to 50% of the cost of software dev elopment. Despite anecdotal evidence that functional programs require somewhat less testing ('Once it type-c hecks, it usually works'), in practice it is still a major part of functional program development.

The cost of testing motivates efforts to automate it, wholly monad are hard to test), and so testing can be done at a fine grain.

A testing tool must be able to determine whether a test is passed or failed; the human tester must supply an automatically checkable criterion of doing so. We have chosen to use formal specifications for this purpose. We have designed a simple domain-specific language of *testable specifications* which the tester uses to define expected properties of the functions under test. QuickCheck then checks that the properties hold in a large number of cases. The specification language is embedded in Haskell using the class system. Properties are normally written in the same module as the functions they test, where they serve also as checkable documentation of the behaviour of the code.

A testing tool must also be able to generate test cases automatically. We have chosen the simplest method, random testing [11], whic hcompetes surprisingly favourably with systematic methods in practice. Ho wever, it is meaningless to talk about random testing without discussing the distribution of test data. Random testing is most effective when the distribution of test data follows that of actual data, but when testing reuseable code units as opposed to whole systems this is not possible, since the distribution of actual


https://dl.acm.org/doi/10.1145/351240.351266

# Property-based Testing in Practice

- 30 interviews with developers at Jane Street
- Insight: use it primarily for testing components of complex systems
  - Especially in “high-leverage” scenarios, where properties are easy to identify and test
- Expect tests to provide greater confidence than conventional example-based unit tests
- Another benefit: used to communicate the specification

## Property-Based Testing in Practice


Harrison Goldstein
University of Pennsylvania
Philadelphia, PA, USA
hgo@seas.upenn.edu

Joseph W. Cutler
University of Pennsylvania
Philadelphia, PA, USA
jwc@seas.upenn.edu

Daniel Dickstein
Jane Street
New York, NY, USA
ddickstein@janestreet.com

Benjamin C. Pierce
University of Pennsylvania
Philadelphia, PA, USA
bcpierce@seas.upenn.edu

Andrew Head
University of Pennsylvania
Philadelphia, PA, USA
head@seas.upenn.edu



**ABSTRACT**

Property-based testing (PBT) is a testing methodology where users write executable formal specifications of software components and an automated harness checks these specifications against many automatically generated inputs. From its roots in the QuickCheck library in Haskell, PBT has made significant inroads in mainstream languages and industrial practice at companies such as Amazon, Volvo, and Stripe. As PBT extends its reach, it is important to understand how developers are using it in practice, where they see its strengths and weaknesses, and what innovations are needed to make it more effective.

We address these questions using data from 30 in-depth interviews with experienced users of PBT at Jane Street, a financial technology company making heavy and sophisticated use of PBT. These interviews provide empirical evidence that PBT's main strengths lie in testing complex code and in increasing confidence beyond what is available through conventional testing methodologies, and, moreover, that most uses fall into a relatively small number of high-leverage idioms. Its main weaknesses, on the other hand, lie in the relative complexity of writing properties and random data generators and in the difficulty of evaluating their effectiveness. From these observations, we identify a number of potentially high-impact areas for future exploration, including performance improvements, differential testing, additional high-leverage testing scenarios, better techniques for generating random input data, test-case reduction, and methods for evaluating the effectiveness of tests.


Python's Hypothesis framework [37] had an estimated 500K users in 2021 according to a JetBrains survey [32]. Still, there is plenty of room for growth. Half a million Hypothesis users represent only 4% of the total Python user base, whereas the Hypothesis maintainers estimate [15] that the “addressable market” is at least 25%. (For comparison, the most popular testing framework, pytest, has 50% market share.)

To help move PBT toward wider adoption, the research community (ourselves included) needs to better understand the practical strengths and weaknesses of PBT and the places where further technical advances are required. Existing work in the software engineering literature has studied how other bug-finding tools are used in practice (see §6), but PBT offers a unique set of tools and warrants its own investigation. Accordingly, we interviewed PBT users at Jane Street, a financial technology firm that makes significant use of PBT, to learn how they use PBT, where they see its value, and in what ways they think it might be improved. Concretely, we aimed to answer two main questions:

**RQ1:** What are the characteristics of a successful and mature PBT culture at a software company?

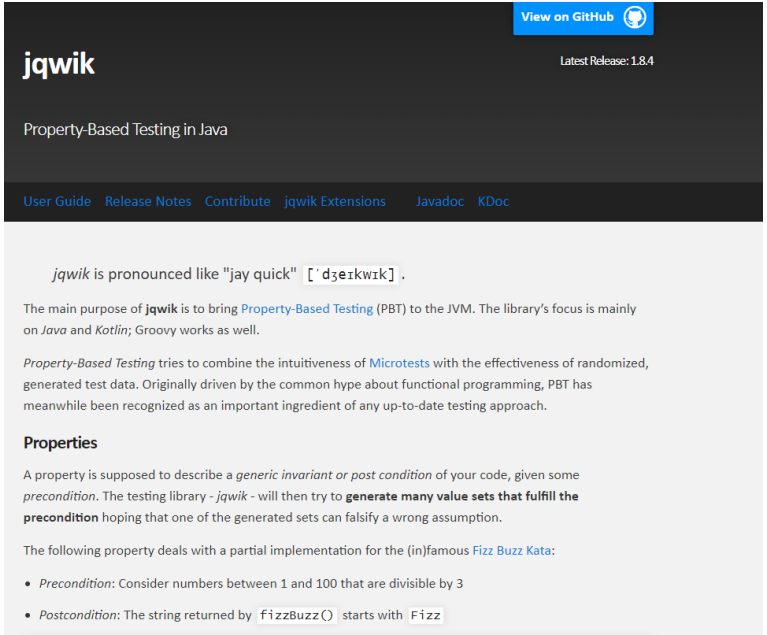
**RQ2:** Are there opportunities for future work in the PBT space that are motivated by the needs of real developers?

The first question aims both to offer guidance for engineers and managers considering whether PBT might fit well in their organizations and to provide a basis for evaluating and comparing PBT technologies. The second question aims to help shape further re-


https://dl.acm.org/doi/abs/10.1145/3597503.3639581

# jqwik

- Jqwik is one of many property-based testing frameworks
- For Java
- Interfaces might differ significantly between different frameworks (unlike for unit testing frameworks)



View on GitHub

jqwik

Latest Release: 1.8.4

Property-Based Testing in Java

User Guide Release Notes Contribute jqwik Extensions Javadoc KDoc

*jqwik* is pronounced like "jay quick" [ˈdʒeɪkwɪk].

The main purpose of **jqwik** is to bring Property-Based Testing (PBT) to the JVM. The library's focus is mainly on *Java* and *Kotlin*; Groovy works as well.

*Property-Based Testing* tries to combine the intuitiveness of Microtests with the effectiveness of randomized, generated test data. Originally driven by the common hype about functional programming, PBT has meanwhile been recognized as an important ingredient of any up-to-date testing approach.

**Properties**

A property is supposed to describe a *generic invariant or post condition* of your code, given some *precondition*. The testing library - *jqwik* - will then try to **generate many value sets that fulfill the precondition** hoping that one of the generated sets can falsify a wrong assumption.

The following property deals with a partial implementation for the (in)famous Fizz Buzz Kata:

- *Precondition*: Consider numbers between 1 and 100 that are divisible by 3
- *Postcondition*: The string returned by `fizzBuzz()` starts with `Fizz`

https://jqwik.net/

# Property-based Testing

- Let’s assume we want to test Math.abs(x)
- Inputs any integer
- Property: |x| > 0

$$|x| = \begin{cases} x, & \text{if } x \geq 0 \\ -x, & \text{if } x < 0. \end{cases}$$

# Property-based Testing

```
@Property
void positiveAbs(@ForAll int val) {
    assertTrue(Math.abs(val) >= 0);
}
```

# Property-based Testing

```
timestamp = 2024-03-17T15:08:01.138157700, TestAbs:positiveAbs =
  java.lang.AssertionError

                                |-----------------------jqwik-----------------------
tries = 149                     | # of calls to property
checks = 149                    | # of not rejected calls
generation = RANDOMIZED         | parameters are randomly generated
after-failure = SAMPLE_FIRST    | try                                          eed
when-fixed-seed = ALLOW         | fixi
edge-cases#mode = MIXIN         | edge
edge-cases#total = 9            | # of
edge-cases#tried = 3            | # of
seed = -9208066553652147292     | random seed to reproduce generated values

Sample
------
  arg0: -2147483648
```

-2147483648 is an input that falsifies the property

# Property-based Testing

```
timestamp = 2024-03-17T15:08:01.138157700, TestAbs:positiveAbs =
  java.lang.AssertionError

tries = 149
checks = 149
generation = RANDOMIZED
after-failure = SAMPLE_FIRST
when-fixed-seed = ALLOW
edge-cases#mode = MIXIN          | edge cases are mixed in
edge-cases#total = 9             | # of all combined edge cases
edge-cases#tried = 3             | # of edge cases tried in current run
seed = -9208066553652147292      | random seed to reproduce generated values

Sample
------
  arg0: -2147483648
```

Generic boundary values:

-2, -1, 0, 2, 1, -2147483648, -2147483647, 2147483647, 2147483646

# Property-based Testing

- Constrain the input domain
- Frameworks like jqwik provide various ways how to adapt existing generators or create new ones

```
@Property
void positiveAbs(@ForAll @IntRange(min = Integer.MIN_VALUE + 1, max =
Integer.MAX_VALUE) int val) {
    assertTrue(Math.abs(val) >= 0);
}
```

# Property-based Testing: CountWords

```
CountWords.count("Surprising storms stress restless riders.")  4
```

Idea: the sum of return values when applying count on individual words should be equal to applying count on the complete input

```
CountWords.count("Surprising")   0
CountWords.count("storms")       1
CountWords.count("stress")       1
CountWords.count("restless")     1
CountWords.count("riders.")      1
                                 ---
                                 4
```

# Property-based Testing

- Generators often need careful consideration
- For random strings (and the method from the testing lecture), it us unlikely that the method returns a non-zero value

```
@Property
void countWordsNaive(@ForAll String val) {
    assertEquals(CountWords.count(val),
        Stream.of(val.split(" ")).mapToInt(s ->
CountWords.count(s)).sum());
}
```

# Property-based Testing

- Solution: override generator to generate more meaningful inputs

```
@Property
void countWordsBetter(@ForAll("countWordStrings") String val) {
    assertEquals(CountWords.count(val), Stream.of(val.split(" ")).mapToInt(s ->
        CountWords.count(s)).sum());
}

@Provide
Arbitrary<String> countWordStrings() {
    return Arbitraries.strings().withCharRange('a', 'z').whitespace()
        .ofMinLength(0).ofMaxLength(100);
}
```

# Task

How can we apply property-based testing to Google Translate?

# Property: Round-trip Translation Similarity

I love learning about software engineering

↓ English to Chinese

我喜欢学习软件工程

↓ Chinese to English

I like studying software engineering

Property to check: round-trip translation results in a semantically similar sentence

# Property: Idempotency

I love learning about software engineering

English to Chinese

我喜欢学习软件工程

I love learning about software engineering

English to Chinese

我喜欢学习软件工程

Property to check: Idempotency; translating the same text multiple times should result in identical translations

# Summary and Takeaways

- Property-based testing: checks invariants, called properties
- Generalization of differential testing and metamorphic testing
- Example-based techniques are simpler and require less creativity to automate
- Property-based testing is useful for complex functionality and if there is uncertainty whether example-based tests are sufficient